



PathGuard-AI — Enhanced ML Architecture Plan

Your Hardware

Component	Spec
GPU	NVIDIA RTX 5050 Laptop — 8 GB VRAM (Blackwell arch, CUDA 13.4)
CPU	Intel i5-13420H — 8 cores / 12 threads
RAM	16 GB DDR5
iGPU	Intel UHD (secondary)

IMPORTANT

RTX 5050 with 8 GB VRAM is a **serious** training-capable GPU. We can train YOLOv8-Small/Medium models directly on-device, run MiDaS depth estimation at 30 FPS, and do real-time inference with TensorRT acceleration.

Original Stack → Enhanced AI/ML Stack

Layer	Original (CPU-only)	Enhanced (RTX 5050 GPU)
Detection Model	YOLOv8-Nano ONNX (pre-trained, frozen)	YOLOv8s/m custom-trained on road defects, TensorRT FP16 optimized
Training	None	Full Ultralytics training pipeline + active learning
Depth Estimation	Intensity heuristic (grayscale drop)	MiDaS v3.1 DPT-Hybrid monocular depth + stereo fusion
Defect Classification	Binary (defect/no-defect)	7-class: pothole, longitudinal_crack, transverse_crack, alligator_crack, rutting, raveling, patching
Segmentation	Contour-based	YOLOv8-Seg instance segmentation for precise area measurement
Inference Runtime	ONNX Runtime CPU	TensorRT FP16 on RTX 5050 (4-10× faster)
Data Augmentation	None	Albumentations: rain, fog, night, glare simulation
Annotation	Manual	Auto-annotation with SAM2 + human review loop
Experiment Tracking	None	MLflow local server
Severity Model	Rule-based only	Hybrid: ML confidence + rule-based IMU fusion

Enhanced Module Architecture



7-Class Defect Taxonomy

Class ID	Defect Type	MoRTH Reference	Visual Signature
0	Pothole	IRC:82-2015 §3.1	Dark depression, irregular edges
1	Longitudinal Crack	IRC:82-2015 §3.2	Linear crack parallel to road center
2	Transverse Crack	IRC:82-2015 §3.3	Linear crack perpendicular to travel
3	Alligator Crack	IRC:82-2015 §3.4	Interconnected crack network
4	Rutting	IRC:82-2015 §3.5	Linear depression in wheel path
5	Raveling	IRC:82-2015 §3.6	Surface aggregate loss, rough texture
6	Patching	IRC:82-2015 §3.7	Previous repair area, different texture

Training Pipeline Details

Phase 1: Data Collection

- Capture dashcam footage while driving
- Extract frames at 2 FPS (avoid redundant frames)
- GPS-tag each extracted frame

Phase 2: Auto-Annotation with SAM2

- Run Segment Anything Model 2 on extracted frames
- Use YOLOv8-pretrained as prompt generator for SAM
- Generate instance masks automatically
- Human reviews and corrects in Label Studio

Phase 3: Training on RTX 5050

- **Model:** YOLOv8s (11.2M params) — fits 8 GB VRAM with batch=16
- **Dataset:** Custom annotated road defect images
- **Augmentation:** Rain, fog, night, shadow, motion blur, perspective warp
- **Epochs:** 100-300 with early stopping

- **Resolution:** 640×640 (training), 1280×720 (inference)
- **Mixed Precision:** FP16 automatic on RTX 5050

Phase 4: Export & Deploy

- Export trained model to TensorRT FP16 engine
- Optimized for RTX 5050 Blackwell architecture
- Expected inference: **2-4ms per frame** (250+ FPS potential)

MiDaS Depth Estimation

Instead of the naive grayscale-intensity depth heuristic, we use **MiDaS v3.1 DPT-Hybrid**:

- Produces dense per-pixel relative depth maps
- We calibrate with known camera height to get absolute depth
- Combined with detected bounding boxes → accurate pothole depth
- Runs at **30+ FPS on RTX 5050** with FP16

Files to Create/Modify

File	Purpose
pathguard/ml/trainer.py	YOLOv8 training pipeline with Ultralytics
pathguard/ml/dataset.py	Dataset management, YOLO format conversion
pathguard/ml/augmentation.py	Albumentations augmentation pipeline
pathguard/ml/auto_annotate.py	SAM2-assisted auto-annotation
pathguard/ml/depth_estimator.py	MiDaS monocular depth estimation
pathguard/ml/tensorrt_engine.py	TensorRT inference optimization
pathguard/ml/experiment.py	MLflow experiment tracking
pathguard/vision.py	Enhanced with GPU inference + multi-class
requirements.txt	Updated with PyTorch, Ultralytics, etc.
config/default_config.yaml	GPU/training config sections
scripts/setup_gpu.ps1	One-click GPU environment setup